

PROJECT REPORT

Detection of AI-Generated Images Using Deep Learning

Implementation and evaluation on the CIFAKE benchmark

Submitted by

Manish Kumar

Keywords: AI-generated images, synthetic image detection, CIFAKE, convolutional neural network, ResNet18, Stable Diffusion

September 2026

Abstract

The rapid progress of text-to-image diffusion models has made it possible to produce pictures that most people cannot tell from photographs. This weakens the evidential value of images and creates risks in misinformation, fraud and impersonation. This project studies a controlled version of the resulting detection problem using CIFAKE, a public benchmark of 120,000 images: 60,000 real photographs from CIFAR-10 and 60,000 synthetic images generated with Stable Diffusion v1.4 for the same ten classes [1].

The work re-implements the published CIFAKE classifier in PyTorch and extends it with a deeper ResNet18 adapted to 32×32 inputs. Training was carried out on Google Colab with a Tesla T4 GPU. The 100,000-image training portion was divided into 90,000 training and 10,000 validation images with a stratified, seeded split, and the official 20,000-image test portion was set aside. The best model state, chosen by validation loss, was saved to Google Drive, and a Gradio interface was added for single-image inference.

On the 10,000-image validation set the compact CNN reached 95.68% accuracy, 96.44% precision, 94.86% recall, an F1-score of 95.64% and a ROC-AUC of 0.9912, while ResNet18 reached a peak validation accuracy of 97.77%. These are validation results for a single generator and a single resolution and should not be read as a general accuracy for AI-image detection. The report therefore places them beside recent work on cross-generator generalisation and real-world robustness, and closes with a roadmap for out-of-distribution testing, Grad-CAM analysis, frequency-domain evidence and provenance signals.

Keywords: AI-generated images; synthetic image detection; CIFAKE; convolutional neural network; ResNet18; Stable Diffusion; explainability.

Contents

1	Introduction	1
1.1	Project Background	1
1.2	Problem Statement	2
1.3	Objectives	3
1.4	Scope of the Project	3
1.5	Expected Outcomes	4
1.6	Organisation of the Report	4
2	System Overview	5
2.1	Existing System	5
2.2	Proposed System	6
2.3	System Modules	6
2.4	Overall Workflow	7
3	Literature Survey	9
3.1	Research Papers Reviewed	9
3.2	Existing Solutions	10
3.3	Comparative Analysis	11
3.4	Research Gap	12
4	Technologies Used	13
4.1	Programming Languages	13
4.2	Frameworks and Libraries	13
4.3	Development Tools	13
4.4	Key Technical Concepts	14
5	Architecture and Design	15
5.1	System Architecture Diagram	15
5.2	Input and Preprocessing Design	16
5.3	Model Architectures	17
5.4	Workflow Explanation	18
6	Implementation	20
6.1	Dataset Acquisition and Verification	20
6.2	Reproducible Stratified Split	20
6.3	Data Pipeline	21
6.4	Model Implementation	21
6.5	Training Procedure with Early Stopping and Checkpointing	21
6.6	Evaluation Utilities	22
6.7	Persistence and Resume	23
6.8	Interactive Inference with Gradio	23
7	Results and Discussion	24
7.1	Screenshots	24
7.2	Output Analysis	27
7.3	Performance Evaluation	29

7.4	Testing Results	30
7.5	Discussion and Limitations	31
8	Challenges and Learning Outcomes	32
8.1	Challenges Faced	32
8.2	Solutions Implemented	32
8.3	Technical Skills Learned	33
9	Future Enhancements	34
9.1	Proposed Improvements	34
9.2	Additional Features	35
9.3	Scalability Opportunities	35
10	Conclusion	37
10.1	Summary of Work Completed	37
10.2	Achievements	37
10.3	Project Impact	37
10.4	Final Remark	37
	References	38
	Epoch-wise Training Records	40

List of Figures

Figure 1.1	Real images from the CIFAR-10 portion of CIFAKE.	2
Figure 1.2	Synthetic images generated with Stable Diffusion v1.4 for the ten classes.	2
Figure 2.1	End-to-end workflow of the project.	8
Figure 3.1	Photorealistic reflections and lighting in generated images.	11
Figure 3.2	Visual defects still found in some generated images.	11
Figure 5.1	System architecture of the implemented CIFAKE detector.	15
Figure 5.2	What the network receives: full-size images reduced to 32×32	16
Figure 5.3	Architecture of PaperCNN with tensor shapes and parameter counts.	17
Figure 5.4	ResNet18 adapted to 32×32 inputs; modified parts are outlined in red.	18
Figure 6.1	Dataset organisation and the training, validation and test split.	21
Figure 6.2	Training loop with best-model checkpointing and early stopping.	22
Figure 7.1	Notebook output: runtime, dataset verification and split sizes.	24
Figure 7.2	Visual sanity check produced by the notebook.	25
Figure 7.3	Notebook output: PaperCNN training log.	25
Figure 7.4	Notebook output: ResNet18 training log.	26
Figure 7.5	Notebook output: checkpoint reload and validation metrics.	26
Figure 7.6	PaperCNN confusion matrix on the 10,000-image validation set.	26
Figure 7.7	Notebook output: launch of the Gradio interface.	27
Figure 7.8	Training behaviour of PaperCNN (top) and ResNet18 (bottom).	28
Figure 7.9	Validation accuracy of the published study and the two notebook models.	29
Figure 9.1	Grad-CAM overlays and heat maps from the original CIFAKE study.	34
Figure 9.2	Proposed evidence-fusion extension of the current classifier.	36

List of Tables

Table 1.1	Composition of the CIFAKE dataset.	1
Table 2.1	The published CIFAKE study compared with this project.	6
Table 2.2	System modules and their status.	7
Table 3.1	Research papers and standards reviewed.	9
Table 3.2	Detection families compared.	12
Table 4.1	Libraries and their role in the project.	13
Table 4.2	Development environment and tools.	14
Table 5.1	Summary of the two models.	18
Table 6.1	Training settings.	22
Table 7.1	Dataset verification recorded by the notebook.	25
Table 7.2	Confusion-matrix analysis for PaperCNN (REAL is positive).	28
Table 7.3	Validation metrics recorded for PaperCNN.	29
Table 7.4	Published and measured results.	30
Table 7.5	Model size and training cost recorded in the notebook.	30
Table 7.6	Verification and testing summary.	30
Table 8.1	Main challenges and their effect.	32
Table 8.2	Skills developed during the project.	33
Table 9.1	Enhancement roadmap.	36
Table A.1	PaperCNN, 18 epochs (early stopping).	40
Table A.2	ResNet18, 14 epochs (early stopping).	41

Introduction

Why detecting synthetic images matters, and how CIFAKE turns it into a controlled learning problem

1.1 Project Background

Not long ago, images produced by software were easy to recognise. Distorted anatomy, smeared texture and inconsistent lighting gave them away. Latent diffusion models have largely removed those cues. A model such as Stable Diffusion starts from random noise and removes it step by step, guided by a text prompt, and it does so in a compressed latent space rather than directly on pixels [3], [4]. The output can include convincing depth of field, water reflections and motion blur, and it takes only seconds to produce [1].

The consequences reach well beyond art. Bird and Lotfi point to fake news, fabricated evidence of an alibi or a crime, and attacks on face and signature verification as realistic misuses of synthetic images [1]. Human judgement is not a dependable safeguard. The GenImage benchmark cites a study in which people separated real from AI-generated images with only 61.3% accuracy [12], and earlier work had already shown that people struggle to notice conventionally edited photographs [23]. Checking authenticity is therefore a task in which machines have to assist people.

CIFAKE [1] offers a compact and transparent setting in which to study that task. It pairs 60,000 real images from CIFAR-10 [2] with 60,000 synthetic images generated by Stable Diffusion v1.4 for the same ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. Every image is a 32×32 RGB picture. Because both halves cover the same semantic content, a classifier cannot succeed by recognising what is shown. It has to learn what separates a photograph from a generated image.

Table 1.1 Composition of the CIFAKE dataset.

Class	Source	Training	Testing	Total
REAL (label 1)	CIFAR-10 photographs [2]	50,000	10,000	60,000
FAKE (label 0)	Stable Diffusion v1.4, generated at 512 px and resized to 32 px [1]	50,000	10,000	60,000
Total		100,000	20,000	120,000



Figure 1.1 Real images from the CIFAR-10 portion of CIFAKE.

Source: Bird and Lotfi [1], Fig. 1; original dataset by Krizhevsky and Hinton [2].

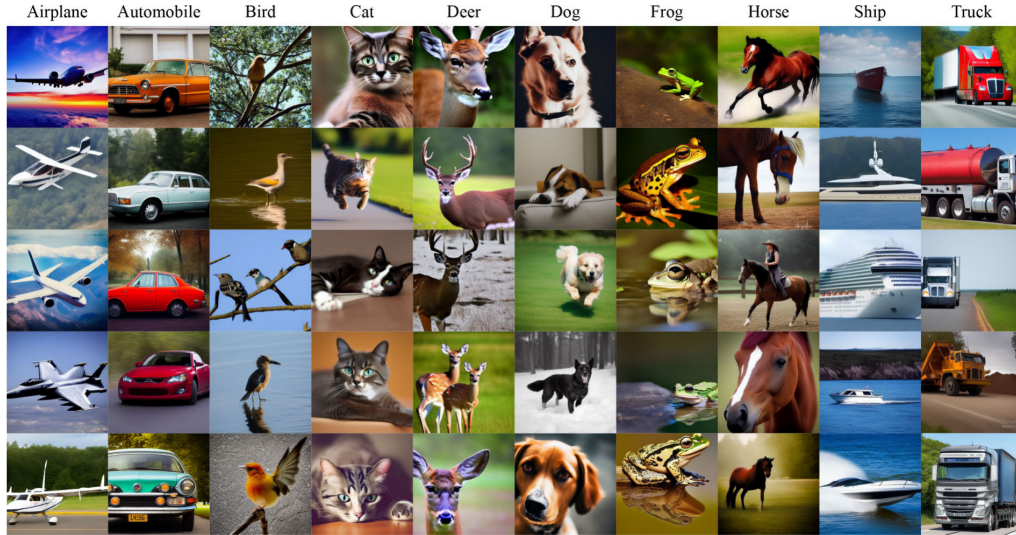


Figure 1.2 Synthetic images generated with Stable Diffusion v1.4 for the ten classes.

Source: Bird and Lotfi [1], Fig. 2. Shown at generation resolution; the dataset stores 32×32 versions.

The original study also reports an interesting result about what the classifier looks at. Using Grad-CAM, the authors found that the depicted object often carries little of the evidence. The decision rests on small imperfections, frequently in the background, whereas real images are recognised in a more holistic way [1]. This shapes how the present report treats its numbers. A high score shows that the classifier has found some regularity that separates the two sets. It does not show that the regularity is one that a person, or a different generator, would share.

1.2 Problem Statement

Given a 32×32 RGB image, decide whether it is a real photograph (label REAL = 1) or an image produced by a latent diffusion model (label FAKE = 0), and output a probability that supports the decision.

The problem has two parts. The classifier must reach high accuracy on the benchmark. Just as importantly, the project must be able to show that the reported accuracy is a fair description of what was measured: the data were verified, the splits do not leak, the metrics expose the type of errors being made, and the limits of the result are stated openly.

1.3 Objectives

1. Download CIFAKE programmatically and verify the expected class counts (50,000 / 50,000 for training and 10,000 / 10,000 for testing) before any training starts.
2. Create a reproducible 90/10 stratified training and validation split from the training folder while keeping the official test folder untouched.
3. Re-implement in PyTorch the compact CNN reported in the CIFAKE paper [1] and train it on a GPU.
4. Add a deeper ResNet18 [15], adapted to 32×32 inputs, as a comparison model under identical data conditions.
5. Evaluate with accuracy, precision, recall, F1-score, ROC-AUC and a confusion matrix, and analyse the error profile rather than a single number.
6. Make the workflow survive Google Colab runtime resets by saving the best model state to Google Drive.
7. Provide an interactive single-image inference interface with Gradio.
8. Relate the results to recent research and define a realistic roadmap for cross-generator, robustness and explainability studies.

1.4 Scope of the Project

Within scope

- Binary REAL versus FAKE classification of 32×32 RGB images from the CIFAKE benchmark.
- Two architectures: the compact CNN from the paper (referred to as PaperCNN) and a modified ResNet18.
- Validation-based evaluation, a reproducible Colab notebook and a demonstration interface.

Outside scope

- Detection of images from other generators, at other resolutions, or after compression, resizing or screenshots.
- Attribution of an image to a specific generator, and any legal or forensic certification of authenticity.
- Training or modifying the generative model itself; the dataset is used exactly as published.
- Provenance and watermarking systems, which are discussed as complementary evidence but not implemented.

1.5 Expected Outcomes

- A documented, reproducible notebook that downloads, verifies, trains, evaluates and serves the model.
- Two trained detectors with checkpoints stored on Google Drive.
- Auditable quantitative evidence: metrics computed from a confusion matrix that can be checked by hand.
- A clear statement of what the numbers do and do not show, supported by recent literature.
- A concrete roadmap for extending the baseline into a more realistic forensic system.

1.6 Organisation of the Report

Chapter 2 describes the existing approaches, the proposed system and its workflow. Chapter 3 surveys the literature and identifies the research gap. Chapter 4 lists the technologies used. Chapter 5 presents the architecture and design, and Chapter 6 the implementation. Chapter 7 reports and discusses the results, Chapter 8 the challenges and learning outcomes, Chapter 9 the future enhancements, and Chapter 10 concludes the report.

System Overview

Existing approaches, the proposed implementation and the end-to-end workflow

2.1 Existing System

Before this project, the practical options for deciding whether an image is synthetic fall into three broad families.

Manual inspection. A reviewer looks for tell-tale artefacts such as malformed hands, garbled text, inconsistent shadows or implausible reflections. Bird and Lotfi show such defects in their own data, including a jet with no cockpit window and a number plate made of pseudo-letters [1], but they also stress that visible defects are becoming rare. Manual review is slow, subjective and hard to scale.

Provenance and watermarking. Instead of analysing pixels, these approaches record or embed information when the image is created. C2PA defines an open standard for attaching signed content credentials that describe the origin and edit history of media [21], and SynthID embeds an imperceptible watermark in content made by supported Google generators [22]. They are strong when present, but they depend on creators, platforms and tools all taking part. An image without credentials is not necessarily fake.

Learned detection. A classifier is trained on labelled real and generated images and outputs a probability of each class. This is the family to which the CIFAKE study belongs, and it is the family implemented in this project.

The published CIFAKE study used TensorFlow on a single RTX 3080 Ti GPU. It trained on 100,000 images and tested on 20,000, and it searched over 36 network topologies: four filter widths and three depths for the convolutional stage, followed by eight dense-layer widths and three depths [1]. It then applied Grad-CAM to a selection of images. The published description concentrates on study design and results. A practical re-implementation must additionally settle engineering questions, such as where checkpoints live when a hosted notebook can disconnect at any time, how data splits are made reproducible, and how a trained model is exposed for use.

2.2 Proposed System

The proposed system keeps the controlled CIFAKE benchmark and the architecture of the original study but rebuilds the experiment as a verified, resumable pipeline in PyTorch. Data are checked before they are used, the training portion is split with a fixed seed, the best model state is written to Google Drive whenever validation loss improves, and the same evaluation code is applied to every model. A ResNet18 is added so that the compact baseline can be compared with a substantially deeper network under the same conditions, and a Gradio interface allows a single image to be tested interactively.

Table 2.1 The published CIFAKE study compared with this project.

Aspect	Published CIFAKE study [1]	This project
Framework	TensorFlow	PyTorch
Compute	RTX 3080 Ti, 12 GB	Google Colab, Tesla T4
Models	36 CNN topologies benchmarked	PaperCNN (best-F1 topology) and ResNet18
Data split	100,000 training, 20,000 testing	90,000 training, 10,000 validation, 20,000 test held out
Metrics	Accuracy, loss, precision, recall, F1	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix
Persistence	Not described	Best checkpoint saved to Google Drive after each improving epoch
Explainability	Grad-CAM applied	Planned; library installed in the notebook
Interface	None described	Gradio single-image demo

2.3 System Modules

The system is organised as a set of small modules with one responsibility each. Keeping them separate means that a new detector can be swapped in without touching data handling or evaluation, which is what makes later comparisons trustworthy.

Table 2.2 System modules and their status.

Module	Responsibility	Status
Dataset acquisition	Download CIFAKE through KaggleHub and locate the REAL and FAKE folders	Implemented
Dataset verification	Check that the four folders contain 50,000 / 50,000 / 10,000 / 10,000 images	Implemented
Split and preprocessing	Stratified 90/10 split; RGB conversion, normalisation, random horizontal flip for training only	Implemented
PaperCNN	Compact convolutional baseline from the CIFAKE paper	Implemented
ResNet18	Deeper comparison model with a CIFAR-style stem	Implemented
Training and checkpointing	Adam optimisation, early stopping, best-model saving to Google Drive	Implemented
Evaluation	Accuracy, precision, recall, F1, ROC-AUC and confusion matrix	Implemented
Interactive inference	Single-image prediction through Gradio	Implemented
Out-of-distribution and robustness tests	Unseen generators; JPEG, blur and resize sweeps	Planned
Explainability and frequency analysis	Grad-CAM and spectral analysis	Planned

2.4 Overall Workflow

The workflow is deliberately linear, and each stage has a check before the next one starts. Figure 2.1 shows the eight stages. Data are downloaded and counted, then split, transformed and passed to the model in mini-batches. After every epoch the model is validated, the best state is saved, and training stops when validation loss has not improved for five consecutive epochs. The saved model is then evaluated on the validation set with probability outputs, so that both threshold-based metrics and the threshold-free ROC-AUC can be computed, and finally it is served through the Gradio interface.

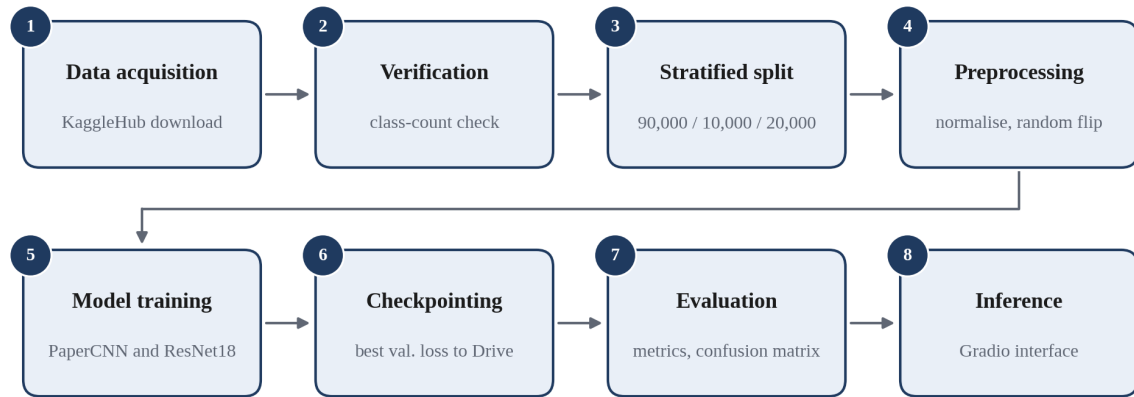


Figure 2.1 End-to-end workflow of the project.

Source: project design, derived from the executed notebook.

Design principle. Each stage produces something that can be checked: class counts, split sizes, per-epoch losses, a saved checkpoint, a confusion matrix. This is what allows the reported numbers to be audited later.

Literature Survey

How AI-generated image detection has developed, and where the CIFAKE setting sits within it

3.1 Research Papers Reviewed

The survey was organised around one question: what happens to a detector when it leaves the exact generator and preprocessing conditions of its training set? The papers below cover the benchmark itself, early and universal detectors, spectral and reconstruction-based methods, large-scale benchmarks, real-world robustness, and provenance.

Table 3.1 Research papers and standards reviewed.

Work	Venue and year	Main contribution	Relevance here
CIFAKE [1]	IEEE Access, 2024	120,000-image benchmark; CNN classifier; Grad-CAM interpretation	Primary reference and baseline
Wang et al. [5]	CVPR, 2020	A classifier trained on one GAN generalises to other CNN generators with suitable augmentation	Early evidence that detectors can transfer
Ojha et al. [6]	CVPR, 2023	Universal detector using features of a large pretrained network	Generalisation beyond the training generator
Corvi et al. [7]	arXiv, 2022	Diffusion images have their own spectral traces, distinct from GAN fingerprints	Motivates frequency analysis
DE-FAKE [8]	arXiv, 2022	Detection and attribution of text-to-image images	Fingerprints in latent-diffusion output
DIRE [9]	ICCV, 2023	Uses the error between an image and its diffusion reconstruction	Generation-process evidence
Synthbuster [10]	IEEE OJSP, 2024	Fourier-domain analysis of diffusion images	Spectral cues independent of content
AEROBLADE [11]	CVPR, 2024	Training-free detection from autoencoder reconstruction error	Alternative to a supervised classifier
GenImage [12]	NeurIPS, 2023	Over one million real and fake pairs; cross-generator and degraded-image tasks	Tests generalisation and robustness
Community Forensics [13]	CVPR, 2025	2.7 million images from 4,803 generator models	Shows the value of generator diversity

Work	Venue and year	Main contribution	Relevance here
RRDataset [14]	ICCV, 2025	17 detectors and 10 vision-language models under internet transmission and re-digitisation	Real-world robustness
C2PA [21], SynthID [22]	Standard and tool	Signed provenance metadata and imperceptible watermarking	Evidence that does not rely on pixels

3.2 Existing Solutions

Spatial-artefact classifiers. The most direct approach trains a CNN to separate real from generated images. Wang et al. showed that a classifier trained on images from one GAN can detect images from several other CNN-based generators when trained with blur and compression augmentation [5]. CIFAKE follows this direct route for latent diffusion images and adds Grad-CAM to see which image regions drive the decision [1]. The strength of the approach is that it optimises the final decision directly. Its weakness is that it can learn shortcuts, such as a generator fingerprint or a resizing artefact, which stop working when the generator changes.

Universal and pretrained-feature detectors. Ojha et al. argued that detectors trained end to end overfit to the artefacts of one generator and instead classified images in the feature space of a large pretrained network, obtaining much better transfer to unseen generators [6]. Community Forensics later supported the same lesson from the data side: detection improves as the number of generator models in the training set grows, even when the models are architecturally similar [13].

Frequency-domain analysis. Corvi et al. showed that diffusion images carry their own spectral traces, different from the fingerprints of GANs [7], and Synthbuster exploits periodic patterns in the Fourier spectrum of diffusion images [10]. Bird and Lotfi also note that latent diffusion output contains digital fingerprints that may be invisible to the eye [1], [8].

Reconstruction-based detection. DIRE reconstructs an image with a pretrained diffusion model and uses the reconstruction error as the feature, because generated images tend to be reconstructed more faithfully than real ones [9]. AEROBLADE applies the same idea to the autoencoder of a latent diffusion model and needs no training at all [11].

Benchmarks that stress detectors. GenImage defines a cross-generator task and a degraded-image task that applies low resolution, JPEG compression and blur [12]. RRDataset goes further by testing 17 detectors and 10 vision-language models on images that have been shared repeatedly across social media and re-digitised, and it adds a study with 192 human participants [14]. Its central message is that performance measured on

clean laboratory images overstates what detectors achieve under real-world conditions.

Provenance and watermarking. C2PA and SynthID answer a different question from the classifiers above. They report how an image was made rather than how it looks [21], [22]. They complement forensic classifiers, and neither replaces the other.

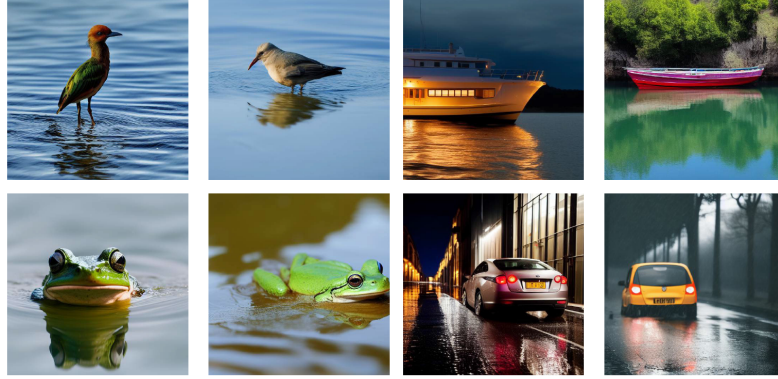


Figure 3.1 Photorealistic reflections and lighting in generated images.

Source: Bird and Lotfi [1], Fig. 4.

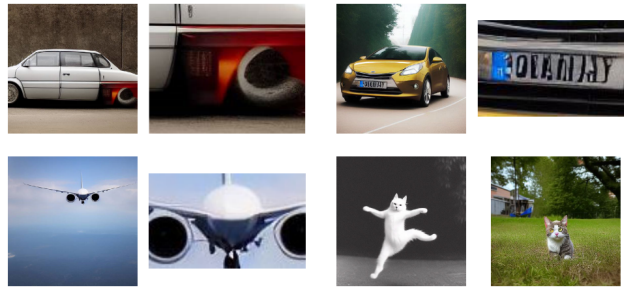


Figure 3.2 Visual defects still found in some generated images.

Source: Bird and Lotfi [1], Fig. 3. Distorted bodywork, a pseudo-text number plate, a jet without a cockpit window and unnatural limbs.

3.3 Comparative Analysis

The families differ in the signal they use, in how well they transfer to new generators, and in what they require from the user. Table 3.2 summarises the comparison.

Table 3.2 Detection families compared.

Approach	Signal used	Main strength	Main limitation
Manual inspection	Visible artefacts	No tooling required	Slow, subjective; defects are disappearing
CNN classifier (CIFAKE)	Learned pixel patterns	Simple, fast, directly optimised	May learn generator-specific shortcuts
Pretrained-feature detector	Features of a large pretrained model	Better transfer to unseen generators	Depends on the pretrained model and the data used
Frequency analysis	Fourier-domain traces	Independent of image content	Traces are weakened by resizing and compression
Reconstruction error	Error against a generative model	Training-free options exist	Requires access to a matching generative model; heavier compute
Provenance and watermarks	Metadata and embedded signals	Strong when present	Needs cooperation from creators and platforms

Against this background the CIFAKE setting has four notable features. It uses a single generator, a fixed and very small resolution, a narrow real-image source, and evaluation on data drawn from the same distribution as the training set. That is exactly what makes it reproducible and suitable for teaching, and exactly what limits how far its scores can be trusted outside the benchmark.

3.4 Research Gap

The gap is not that CIFAKE lacks accuracy. It is that in-distribution accuracy on a small controlled benchmark says little about behaviour on new generators and on images that have been compressed, resized or shared. GenImage and RRDataset were built to expose precisely this difference [12], [14], and Community Forensics shows that generator diversity is a major lever for closing it [13].

Position of this work. This project closes the implementation gap by providing a verified, reproducible and resumable CIFAKE baseline with two architectures and auditable metrics. It does not claim to close the scientific gap of general AI-image detection. The next step, described in Chapter 9, is explicit cross-generator and robustness testing.

Technologies Used

Programming language, machine-learning libraries and the development environment

4.1 Programming Languages

The entire implementation is written in **Python 3**, the language of the Google Colab runtime. Python is used for dataset handling, model definition, the training loop, evaluation and the inference interface. Notebook magic commands (for example `!pip install`) handle package installation. The project guide and documentation are written in **Markdown**.

4.2 Frameworks and Libraries

Table 4.1 Libraries and their role in the project.

Library	Purpose
PyTorch [18]	Tensors, model definition, GPU training, checkpoint saving and loading
Torchvision	Image transforms and the ResNet18 architecture with ImageNet-pretrained weights
scikit-learn [19]	Stratified train and validation split; accuracy, precision, recall, F1, ROC-AUC, confusion matrix
Pillow	Image loading and conversion to RGB
NumPy	Probability and label arrays for evaluation
Matplotlib and Seaborn	Sanity-check images and the confusion-matrix heatmap
KaggleHub	Programmatic download of the CIFAKE dataset [25]
Gradio [20]	Browser-based single-image inference interface
grad-cam (pytorch-grad-cam)	Installed in the notebook for the planned Grad-CAM analysis [16]

4.3 Development Tools

The whole project runs in a hosted notebook environment, which keeps the setup reproducible and free of local installation.

Table 4.2 Development environment and tools.

Tool	Use
Google Colab	Notebook execution on a Tesla T4 GPU, as recorded in the notebook output
Google Drive	Persistent storage for checkpoints and result figures (cifake-project/checkpoints and results)
Kaggle	Public host of the CIFAKE dataset
Jupyter notebook format (.ipynb)	Single notebook organised in sections: setup, data, models, evaluation, demo

4.4 Key Technical Concepts

Convolutional neural network. A CNN applies small learnable filters across an image. Early filters capture edges and textures and deeper layers combine them into more complex patterns. For a detector like CIFAKE the useful evidence may be much smaller than the object itself, which is why a CNN working on local patterns is a natural choice [1].

Binary classification head. The network ends in a single output value, the logit z . The sigmoid function converts it into a probability of the REAL class, and training uses binary cross-entropy computed directly from the logit for numerical stability.

$$p(\text{REAL}) = 1 / (1 + e^{-z})$$

Evaluation metrics. With REAL as the positive class, precision is the share of images predicted REAL that really are REAL, recall is the share of real images that are found, and F1 is their harmonic mean. ROC-AUC measures how well the probability ranks real above fake images across all thresholds.

$$\text{Precision} = TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN) \quad F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

Residual learning. ResNet18 adds the input of a block to its output, so that each block learns a correction rather than a complete transformation, which makes deeper networks easier to train [15].

$$y = F(x) + x$$

Architecture and Design

A modular pipeline from input image to evaluated prediction

5.1 System Architecture Diagram

Figure 5.1 shows the architecture. Everything between the dataset and the user runs inside one Google Colab session on a Tesla T4 GPU. Two external services complete the picture: Kaggle supplies the data, and Google Drive keeps whatever would be expensive to recreate.

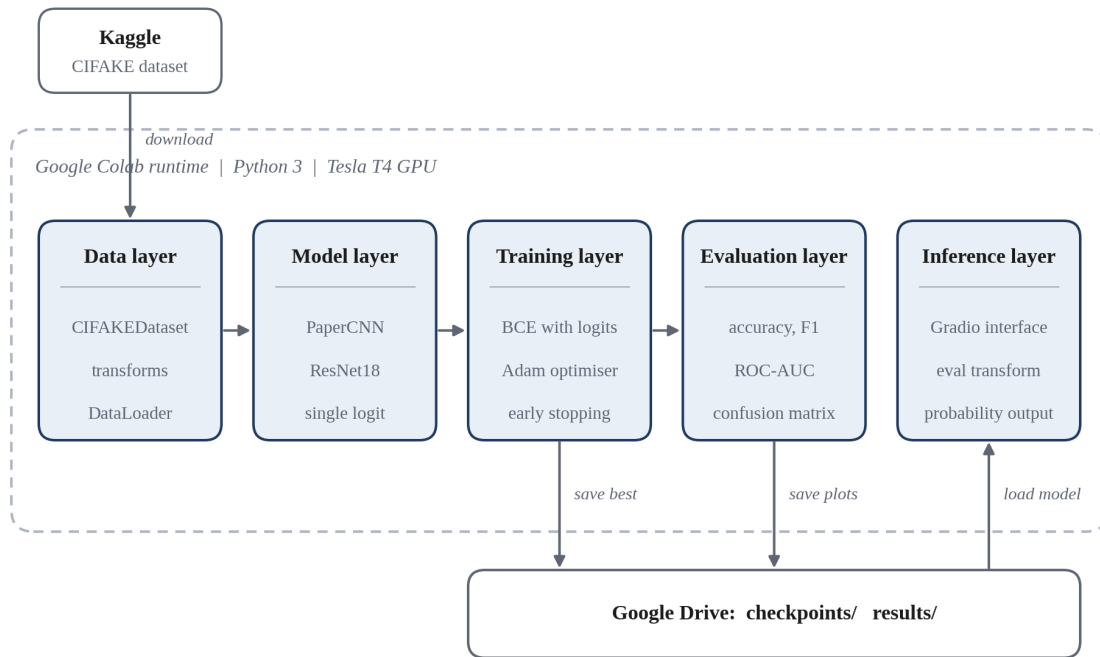


Figure 5.1 System architecture of the implemented CIFAKE detector.

Source: project design, derived from the executed notebook.

- **Data layer.** A custom *CIFAKEDataset* lists the image files of each class and returns a transformed image with its label. Two *DataLoader* objects (training and evaluation) feed mini-batches to the GPU.
- **Model layer.** PaperCNN and ResNet18 share one interface: an image tensor goes in and a single logit comes out. This is what lets one training and evaluation routine serve both.

- **Training layer.** Binary cross-entropy with logits, the Adam optimiser [17] and early stopping on validation loss.
- **Evaluation layer.** Probabilities are stored, so threshold-based metrics and ROC-AUC are computed from the same predictions.
- **Inference layer.** The Gradio interface applies the evaluation transform and reports the probability of each class.
- **Persistence.** Only the checkpoints and result figures are written to Google Drive. The raw dataset is deliberately kept on the local runtime disk, because 120,000 small files are slow to work with on Drive and would consume its quota.

5.2 Input and Preprocessing Design

The detector accepts a 32×32 RGB image. This matches CIFAKE, where the real images are natively 32×32 and the synthetic ones were rendered at 512 pixels and then reduced to 32×32 by bilinear interpolation [1]. Matching the input size keeps the comparison with the published study like for like, but it also removes almost all fine detail. Figure 5.2 illustrates the effect with a real photograph and a generated image taken from the web. These two pictures are not part of CIFAKE; they are shown only to make the reduction visible.

Real photograph (web) | 1546×1213 px



32×32 px, as received by the network



AI-generated image (web) | 512×512 px



32×32 px, as received by the network



The 32×32 versions are enlarged with nearest-neighbour scaling so that individual pixels remain visible.

Figure 5.2 What the network receives: full-size images reduced to 32×32 .

Sources: PyTorch Hub example photograph (github.com/pytorch/hub); Stable Diffusion sample output (github.com/CompVis/stable-diffusion).

Every image is converted to RGB, turned into a tensor and normalised with the channel means (0.4914, 0.4822, 0.4465) and standard deviations (0.2470, 0.2435, 0.2616) commonly used for CIFAR-10. Training images are also flipped horizontally at random.

Validation and test images receive no random augmentation, so that repeated evaluations give identical results. The class labels follow the convention of the original paper: REAL is 1 and FAKE is 0 [1].

5.3 Model Architectures

PaperCNN follows the best-F1 topology reported by Bird and Lotfi [1]: two convolutional blocks of 32 filters, each followed by max-pooling, then a dense layer of 64 units and a single output. Padding is used in the convolutions so that the spatial size is halved only by pooling. The network has 141,345 trainable parameters.

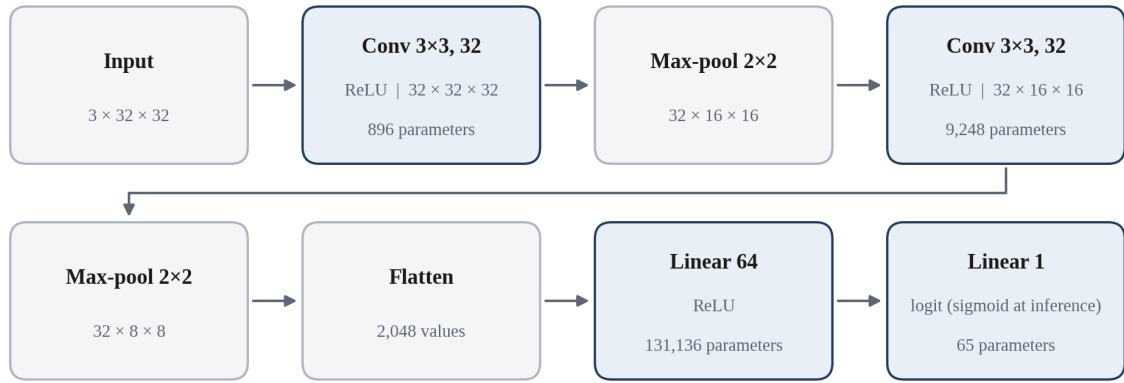


Figure 5.3 Architecture of PaperCNN with tensor shapes and parameter counts.

Source: project implementation; topology after Bird and Lotfi [1], Fig. 5.

ResNet18 [15] is the comparison model. It is initialised with ImageNet weights, but two parts are changed for 32×32 inputs. The original 7×7 stride-2 convolution and the following max-pooling layer are replaced by a 3×3 stride-1 convolution and an identity mapping, so the first stage still sees the full 32×32 resolution. The final fully connected layer is replaced by a single output neuron. The replaced stem and head are trained from scratch. The network has about 11.17 million parameters.

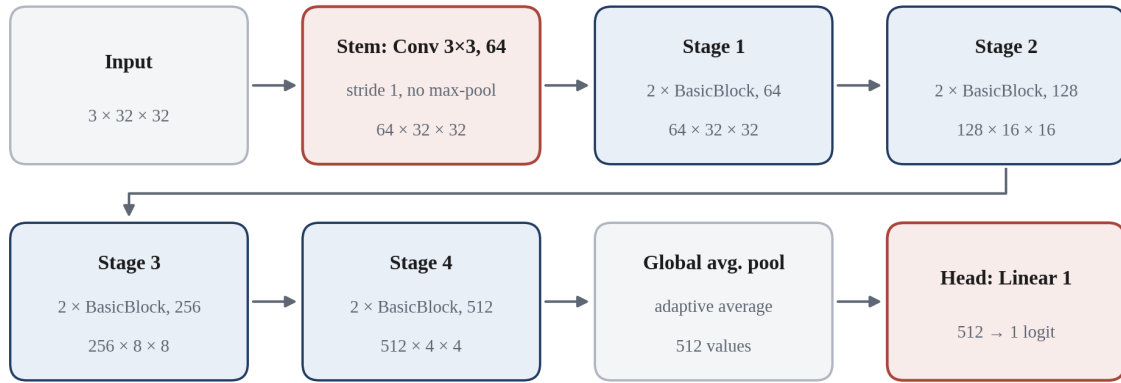


Figure 5.4 ResNet18 adapted to 32×32 inputs; modified parts are outlined in red.

Source: project implementation; architecture after He et al. [15].

Table 5.1 Summary of the two models.

Property	PaperCNN	ResNet18
Input	$3 \times 32 \times 32$	$3 \times 32 \times 32$
Core layers	2 convolutional blocks (32 filters), dense layer of 64	Stem, 4 stages of 2 BasicBlocks each (64 to 512 channels)
Output	1 logit	1 logit
Trainable parameters	141,345	about 11.17 million
Initialisation	Random	ImageNet weights; stem and head from scratch
Learning rate	0.001	0.0005
Epoch budget	20	15

5.4 Workflow Explanation

Training workflow. Each epoch has two phases. In the training phase the model processes shuffled mini-batches of 128 images, the loss is computed from the logits and the Adam optimiser updates the weights. In the validation phase the model, with gradients switched off, processes the 10,000 validation images and the mean loss and accuracy are recorded. If the validation loss is the best so far, the model state, epoch number, accuracy and full history are written to Google Drive. If it has not improved for five consecutive epochs, training stops. The reasoning is that validation loss, not training loss, indicates how well the model will behave on images it has not seen.

Inference workflow. A new image is converted to RGB and passed through the same evaluation transform, the model returns a logit, and the sigmoid converts it into the probability of REAL. The probability of AI-generated is one minus that value. Predictions

are made by thresholding at 0.5, but the probability itself is preserved so that better operating points can be chosen later.

Design principles.

- Separate data handling, training, evaluation and inference so that any part can be replaced independently.
- Persist expensive results (checkpoints) rather than relying on the lifetime of one Colab session.
- Keep the official test set separate from the validation set and label every metric with the data it was computed on.
- Treat future experiments as new evidence channels added beside the baseline, not as replacements for it.

Implementation

Key features implemented in the Colab notebook

This chapter describes the main features of the implementation in the order in which they run in the notebook.

6.1 Dataset Acquisition and Verification

The dataset is downloaded with KaggleHub from the public repository *birdy654/cifake-real-and-ai-generated-synthetic-images* [25]. It is stored on the local Colab disk, and the notebook counts the files in each of the four folders (train and test, REAL and FAKE) before doing anything else. The expected counts are 50,000, 50,000, 10,000 and 10,000, and the executed notebook confirmed all four. This check is cheap and prevents a wrong folder structure from silently producing misleading results.

6.2 Reproducible Stratified Split

The 100,000 training images are divided 90/10 into a training subset and a validation subset using scikit-learn with a stratified split and a fixed random seed of 42. Stratification keeps the two classes balanced in both subsets (45,000 and 45,000 for training, 5,000 and 5,000 for validation). The validation subset is built from a second dataset object that uses the evaluation transform, so that no random augmentation touches it. The official 20,000-image test folder is loaded but kept aside.

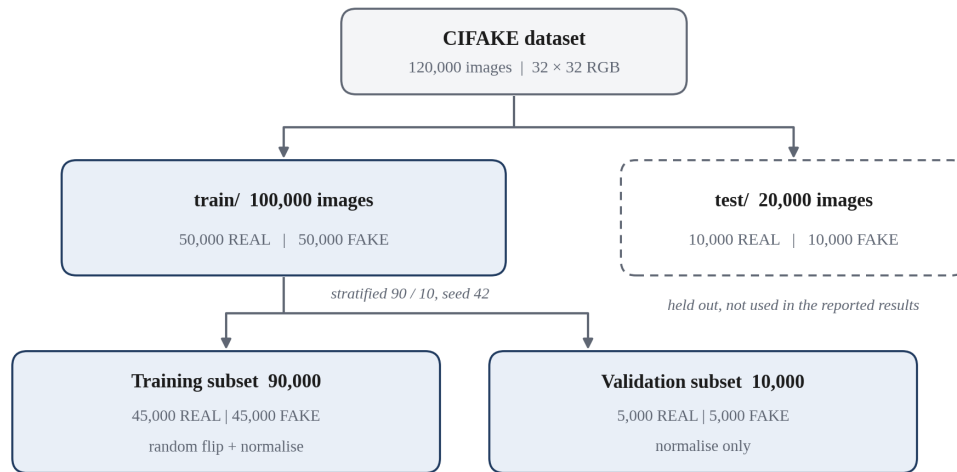


Figure 6.1 Dataset organisation and the training, validation and test split.

Source: project data-preparation design, matching the executed notebook.

6.3 Data Pipeline

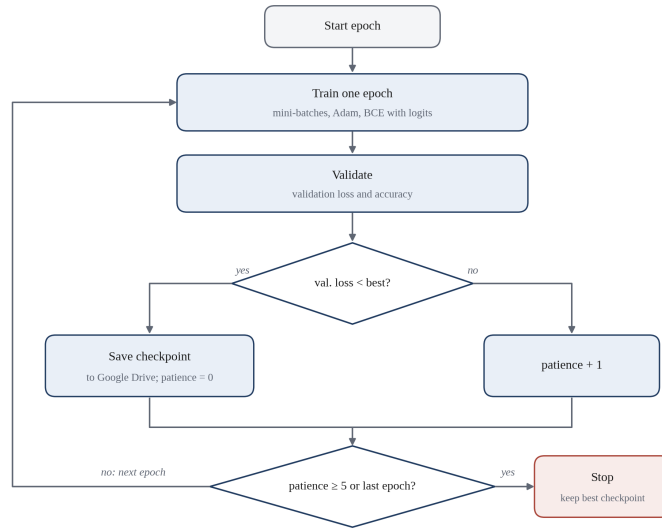
The dataset class opens each file with Pillow, converts it to RGB, applies the transform and returns the image tensor with a float label. Training batches contain 128 images and evaluation batches 256. Two worker processes load data in parallel. More workers were avoided because the Colab virtual machine has few CPU cores and extra workers can slow loading down or crash the session.

6.4 Model Implementation

Both networks are written as PyTorch modules that return a raw logit. PaperCNN is a small sequential model of two convolution, ReLU and max-pooling blocks followed by a flatten operation, a 64-unit dense layer with ReLU and a one-unit output layer. ResNet18 is built from torchvision with ImageNet weights, and its first convolution, max-pooling layer and final layer are replaced as described in Chapter 5. Returning logits rather than probabilities allows the numerically stable loss function to be used during training.

6.5 Training Procedure with Early Stopping and Checkpointing

One training function serves both models. It uses binary cross-entropy with logits and the Adam optimiser, records training loss, validation loss and validation accuracy for every epoch, and applies the checkpointing and early-stopping logic shown in Figure 6.2. Table 6.1 lists the settings used.

**Figure 6.2** Training loop with best-model checkpointing and early stopping.*Source: project implementation.***Table 6.1** Training settings.

Setting	PaperCNN	ResNet18
Optimiser	Adam	Adam
Learning rate	0.001	0.0005
Loss	BCE with logits	BCE with logits
Maximum epochs	20	15
Early-stopping patience	5 epochs (validation loss)	5 epochs (validation loss)
Batch size	128 training, 256 evaluation	128 training, 256 evaluation
Augmentation	Random horizontal flip (training only)	Random horizontal flip (training only)
Split	90/10 stratified, seed 42	90/10 stratified, seed 42
Decision threshold	0.5	0.5

6.6 Evaluation Utilities

A single evaluation routine runs a model over a data loader without gradients and returns the predicted probabilities and true labels. From these, accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix are computed with scikit-learn. Storing probabilities instead of only class predictions is what makes the ROC-AUC calculation possible and leaves room for calibration and threshold studies later.

6.7 Persistence and Resume

Every improving epoch writes a checkpoint containing the model weights, the epoch number, the validation accuracy and the training history to the project folder on Google Drive. A separate cell reloads the checkpoint into a fresh model, so a disconnected session can be resumed by running the setup cells and the load cell alone. The executed notebook demonstrates this: the reloaded PaperCNN reports epoch index 12 and validation accuracy 0.9568, identical to the value saved during training.

6.8 Interactive Inference with Gradio

The final section wraps the trained PaperCNN in a Gradio interface. A user uploads an image, the function converts it to RGB, applies the evaluation transform, runs the model and returns the probabilities of REAL and AI-generated. The notebook launched the interface with a temporary public link. Such links expire, so a permanent deployment would move the app to a hosted service such as Hugging Face Spaces.

Implementation boundary. Cross-generator testing, robustness sweeps, frequency analysis and Grad-CAM are not part of the executed notebook. They appear in this report as planned work only, and no result is claimed for them.

Results and Discussion

Notebook outputs, quantitative evaluation, testing results and the limits of the findings

7.1 Screenshots

This section shows the outputs of the executed notebook. The sanity-check image and the confusion matrix are the notebook's own image outputs. The text outputs are re-typeset from the notebook so that they remain legible in print; their content is unchanged apart from the removal of an emoji, and the identifier of the temporary Gradio link.

```

Notebook output: runtime, dataset verification and data split

# runtime check and Drive mount
GPU available: True
Device: Tesla T4
Drive already mounted at /content/drive; to attempt to forcibly remount, call
  drive.mount("/content/drive", force_remount=True).

# dataset download (KaggleHub)
Downloaded to: /kaggle/input/cifake-real-and-ai-generated-synthetic-images

# class-count verification
train REAL 50000
train FAKE 50000
test REAL 10000
test FAKE 10000

# train / validation / test split
Train: 90000 | Val: 10000 | Test: 20000

# data loaders
DataLoaders ready!

```

Figure 7.1 Notebook output: runtime, dataset verification and split sizes.

Table 7.1 summarises what this output confirms. The GPU was a Tesla T4, all four class folders contained the expected number of images, and the split produced 90,000, 10,000 and 20,000 images.

Table 7.1 Dataset verification recorded by the notebook.

Folder	Expected	Observed
train / REAL	50,000	50,000
train / FAKE	50,000	50,000
test / REAL	10,000	10,000
test / FAKE	10,000	10,000
Training / validation / test subsets	90,000 / 10,000 / 20,000	90,000 / 10,000 / 20,000

**Figure 7.2** Visual sanity check produced by the notebook.

Source: executed notebook (first row of the output figure; the unused empty second row is cropped).

The sanity check confirms that images load, convert and display correctly. Its purpose is to catch colour-channel errors, broken paths and malformed samples before training, not to prove that the classes are separable.

Notebook output: PaperCNN training log (18 epochs)

```
Epoch 1/20 | train_loss 0.3182 | val_loss 0.2173 | val_acc 0.9111 | 170.2s
-> saved best checkpoint (val_acc=0.9111)
Epoch 2/20 | train_loss 0.2037 | val_loss 0.1992 | val_acc 0.9180 | 62.6s
-> saved best checkpoint (val_acc=0.9180)
Epoch 3/20 | train_loss 0.1702 | val_loss 0.1786 | val_acc 0.9287 | 63.4s
-> saved best checkpoint (val_acc=0.9287)
Epoch 4/20 | train_loss 0.1570 | val_loss 0.1805 | val_acc 0.9305 | 63.0s
Epoch 5/20 | train_loss 0.1445 | val_loss 0.1432 | val_acc 0.9439 | 64.2s
-> saved best checkpoint (val_acc=0.9439)
Epoch 6/20 | train_loss 0.1390 | val_loss 0.1362 | val_acc 0.9443 | 63.9s
-> saved best checkpoint (val_acc=0.9443)
Epoch 7/20 | train_loss 0.1294 | val_loss 0.1690 | val_acc 0.9344 | 66.6s
Epoch 8/20 | train_loss 0.1205 | val_loss 0.1294 | val_acc 0.9506 | 65.7s
-> saved best checkpoint (val_acc=0.9506)
Epoch 9/20 | train_loss 0.1147 | val_loss 0.1231 | val_acc 0.9518 | 66.6s
-> saved best checkpoint (val_acc=0.9518)
Epoch 10/20 | train_loss 0.1088 | val_loss 0.1245 | val_acc 0.9525 | 64.9s
Epoch 11/20 | train_loss 0.1010 | val_loss 0.1405 | val_acc 0.9492 | 63.7s
Epoch 12/20 | train_loss 0.0960 | val_loss 0.1366 | val_acc 0.9482 | 65.2s
Epoch 13/20 | train_loss 0.0914 | val_loss 0.1228 | val_acc 0.9568 | 63.9s
-> saved best checkpoint (val_acc=0.9568)
Epoch 14/20 | train_loss 0.0842 | val_loss 0.1257 | val_acc 0.9538 | 65.1s
Epoch 15/20 | train_loss 0.0825 | val_loss 0.1368 | val_acc 0.9529 | 65.6s
Epoch 16/20 | train_loss 0.0777 | val_loss 0.1407 | val_acc 0.9518 | 67.6s
Epoch 17/20 | train_loss 0.0724 | val_loss 0.1251 | val_acc 0.9567 | 65.5s
Epoch 18/20 | train_loss 0.0683 | val_loss 0.1271 | val_acc 0.9565 | 63.3s
Early stopping.
```

Figure 7.3 Notebook output: PaperCNN training log.

Notebook output: ResNet18 training log (14 epochs)

```
Epoch 1/15 | train_loss 0.1555 | val_loss 0.1423 | val_acc 0.9451 | 85.7s
-> saved best checkpoint (val_acc=0.9451)
Epoch 2/15 | train_loss 0.0947 | val_loss 0.1194 | val_acc 0.9551 | 83.7s
-> saved best checkpoint (val_acc=0.9551)
Epoch 3/15 | train_loss 0.0765 | val_loss 0.0888 | val_acc 0.9685 | 87.7s
-> saved best checkpoint (val_acc=0.9685)
Epoch 4/15 | train_loss 0.0638 | val_loss 0.0837 | val_acc 0.9701 | 86.4s
-> saved best checkpoint (val_acc=0.9701)
Epoch 5/15 | train_loss 0.0588 | val_loss 0.0932 | val_acc 0.9691 | 86.6s
Epoch 6/15 | train_loss 0.0469 | val_loss 0.0840 | val_acc 0.9708 | 85.4s
Epoch 7/15 | train_loss 0.0379 | val_loss 0.1264 | val_acc 0.9611 | 85.4s
Epoch 8/15 | train_loss 0.0362 | val_loss 0.0710 | val_acc 0.9771 | 85.4s
-> saved best checkpoint (val_acc=0.9771)
Epoch 9/15 | train_loss 0.0301 | val_loss 0.0677 | val_acc 0.9775 | 85.9s
-> saved best checkpoint (val_acc=0.9775)
Epoch 10/15 | train_loss 0.0263 | val_loss 0.0745 | val_acc 0.9776 | 86.7s
Epoch 11/15 | train_loss 0.0229 | val_loss 0.0739 | val_acc 0.9777 | 84.8s
Epoch 12/15 | train_loss 0.0213 | val_loss 0.0824 | val_acc 0.9738 | 85.5s
Epoch 13/15 | train_loss 0.0194 | val_loss 0.0799 | val_acc 0.9769 | 85.1s
Epoch 14/15 | train_loss 0.0169 | val_loss 0.0770 | val_acc 0.9760 | 85.4s
Early stopping.
```

Figure 7.4 Notebook output: ResNet18 training log.

Notebook output: checkpoint reload and evaluation

```
# reload best checkpoint from Google Drive
Loaded model from epoch 12, val_acc=0.9568

# validation-set evaluation
Accuracy: 0.9568 | Precision: 0.9644 | Recall: 0.9486 | F1: 0.9564 | AUC: 0.9912
```

Figure 7.5 Notebook output: checkpoint reload and validation metrics.

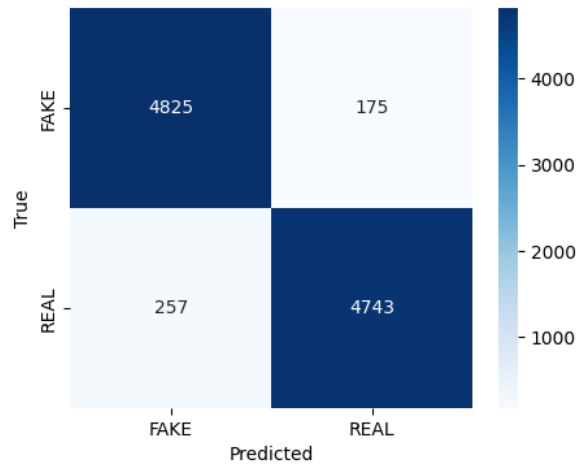


Figure 7.6 PaperCNN confusion matrix on the 10,000-image validation set.

Source: executed notebook. REAL is the positive class.

```
Notebook output: Gradio interface launch (link identifier removed)

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://<id>.gradio.live

This share link is temporary and will last for up to 1 week (best effort).
For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in
the working directory to deploy to Hugging Face Spaces (https://huggingface.co/spaces)
```

Figure 7.7 Notebook output: launch of the Gradio interface.

The public link is temporary; its identifier has been removed.

7.2 Output Analysis

Training behaviour

Figure 7.8 plots the recorded losses and accuracies. PaperCNN improves quickly during the first eight epochs and then plateaus at roughly 95.2 to 95.7% validation accuracy. Its training loss keeps falling, from 0.3182 in the first epoch to 0.0683 in the eighteenth, while validation loss stays near 0.12 to 0.14 and does not improve after epoch 13. This widening gap is the typical sign of a model that has begun to fit its training images more closely than it generalises, and it is the reason for saving the best-validation-loss checkpoint instead of the last one. Early stopping ended training after epoch 18.

ResNet18 starts from a higher point, with 94.51% validation accuracy after the first epoch, and reaches 97.7% by epoch 8. Its validation loss is lowest at epoch 9 (0.0677), where the saved checkpoint records 97.75%. A slightly higher accuracy of 97.77% occurs at epoch 11, but the validation loss there is worse, so that state was not saved. Its training loss falls to 0.0169 by epoch 14, a wider gap than PaperCNN's, yet its validation results are better, which shows that greater capacity and pretrained initialisation helped even at this small image size.

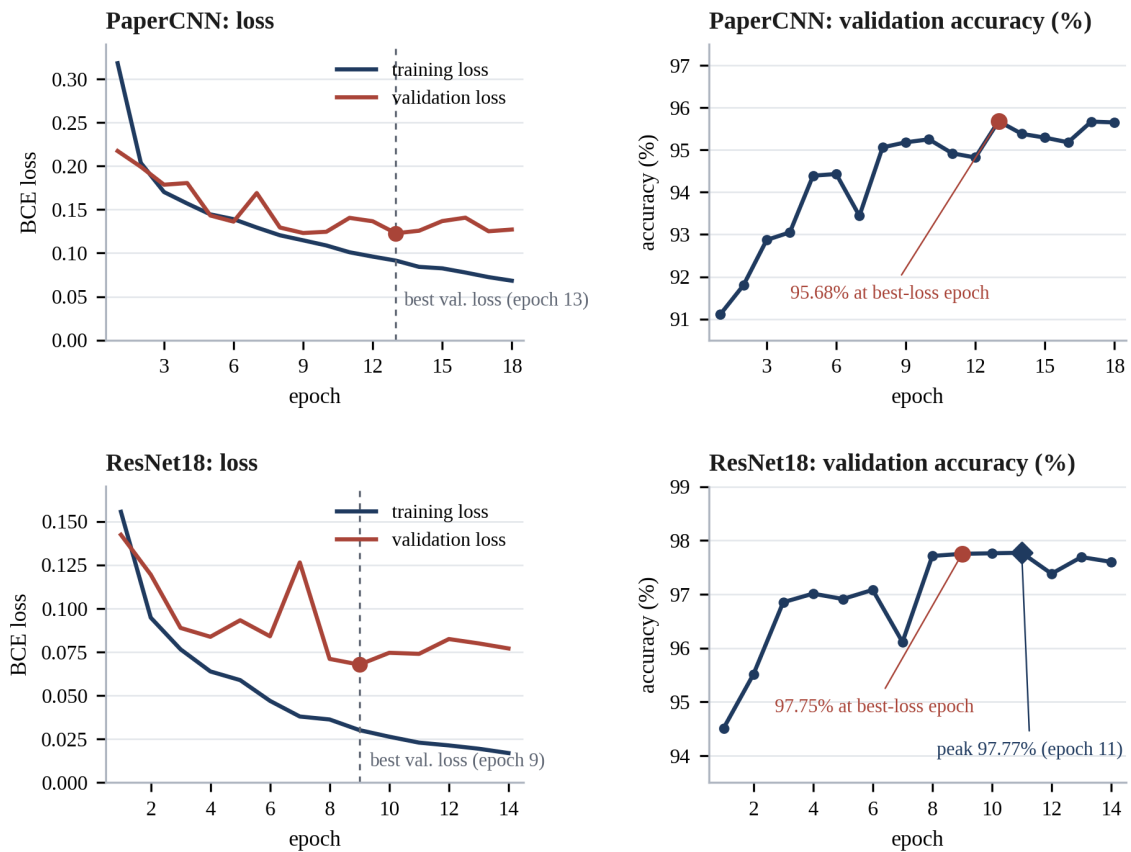


Figure 7.8 Training behaviour of PaperCNN (top) and ResNet18 (bottom).

Source: plotted from the training logs recorded in the executed notebook.

Error profile of PaperCNN

Table 7.2 converts the confusion matrix into error rates. Of 5,000 real validation images, 257 (5.14%) were classified as FAKE. Of 5,000 synthetic images, 175 (3.50%) were classified as REAL. The classifier is therefore somewhat more likely to reject a real photograph than to accept a generated one. This asymmetry matters for the use case discussed in the original paper, where a real image wrongly labelled as AI-generated would falsely accuse its author [1]. A decision threshold above 0.5 for the FAKE label, or a review step for borderline cases, would be a way to control that risk.

Table 7.2 Confusion-matrix analysis for PaperCNN (REAL is positive).

Quantity	Count	Rate	Interpretation
True positives (TP)	4,743	94.86%	Real images correctly recognised (recall)
False negatives (FN)	257	5.14%	Real images labelled FAKE
True negatives (TN)	4,825	96.50%	Fake images correctly recognised (specificity)
False positives (FP)	175	3.50%	Fake images labelled REAL
Total errors	432	4.32%	FN + FP out of 10,000

7.3 Performance Evaluation

All figures below are computed on the 10,000-image validation set. They were verified by hand from the confusion matrix: precision is 4,743 divided by 4,918, and recall is 4,743 divided by 5,000.

Table 7.3 Validation metrics recorded for PaperCNN.

Metric	Value	Meaning in this experiment
Accuracy	95.68%	Correct decisions among 10,000 validation images
Precision	96.44%	Of the images predicted REAL, the share that were REAL
Recall	94.86%	Of the real images, the share that were found
F1-score	95.64%	Harmonic mean of precision and recall
ROC-AUC	0.9912	Probability that a random real image is scored above a random fake one

Comparison with the published study. Figure 7.9 and Table 7.4 place the results beside the original paper. The abstract of the paper quotes 92.98% accuracy, its dense-layer experiments reach up to 93.55%, and its highest F1-score is 0.936 [1]. PaperCNN, which uses the same topology, scores 95.68% accuracy and an F1-score of 0.9564 here, and ResNet18 peaks at 97.77% accuracy. These numbers show that the re-implementation is in the right range and that a deeper network helps. They should not be presented as an improvement over the original work. The evaluation data differ (a 10,000-image split taken from the training folder here, a 20,000-image test portion in the paper), the framework and training details differ, and none of the numbers has been tested on an unseen generator.

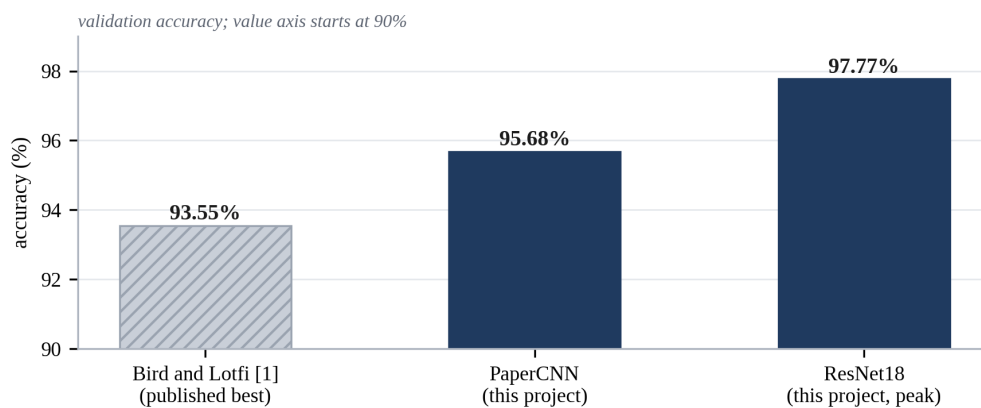


Figure 7.9 Validation accuracy of the published study and the two notebook models.

Sources: Bird and Lotfi [1] (Table 7 maximum) and the executed notebook. Evaluation splits differ, so this is an indicative comparison.

Table 7.4 Published and measured results.

Model	Accuracy	F1-score	Evaluated on
Published best dense stage [1]	93.55% (headline 92.98%)	0.936	20,000-image test portion, reported as validation accuracy
PaperCNN (this project)	95.68%	0.9564	10,000-image validation split
ResNet18 (this project)	97.77% peak (97.75% saved)	not recorded	10,000-image validation split

Computational cost. Table 7.5 shows that ResNet18 has roughly eighty times as many parameters as PaperCNN but needs only about 1.3 times as long per epoch. This suggests that on the Colab machine the input pipeline, which reads 90,000 small image files with two worker processes, rather than the network arithmetic, is the main cost. The first PaperCNN epoch took 170.2 seconds against about 63 seconds for later epochs, which most likely reflects one-off file reading and caching, although the notebook does not record the cause.

Table 7.5 Model size and training cost recorded in the notebook.

Model	Parameters	Epochs run	Best-loss epoch	Time per epoch	Total time
PaperCNN	141,345	18	13	62.6 to 67.6 s (first: 170.2 s)	about 21 min
ResNet18	about 11.17 million	14	9	83.7 to 87.7 s	about 20 min

7.4 Testing Results

The notebook contains no separate test suite. The checks below are the verification steps that are recorded in its output, together with two steps that were not carried out. Reporting the second group is as important as reporting the first.

Table 7.6 Verification and testing summary.

Check	Expected	Observed	Result
GPU availability	CUDA GPU present	Tesla T4	Pass
Dataset counts	50,000 / 50,000 / 10,000 / 10,000	As expected	Pass
Split sizes	90,000 / 10,000 / 20,000	As expected	Pass
Data loaders	Build without error	Ready	Pass
Sample images	Load and display correctly	Eight images rendered	Pass
Early stopping	Stops after 5 epochs without improvement	PaperCNN at epoch 18, ResNet18 at epoch 14	Pass

Check	Expected	Observed	Result
Checkpoint reload	Reproduces saved accuracy	0.9568 after reload	Pass
Metric consistency	Evaluation accuracy equals training-loop accuracy	0.9568 in both	Pass
Confusion-matrix consistency	Precision and recall follow from the counts	96.44% and 94.86% recomputed	Pass
Gradio interface	Launches	Launched with a temporary link; sample predictions not recorded	Launch only
Official test set (20,000)	Final accuracy estimate	Not run	Not done
Unseen generators and image degradation	Robustness estimate	Not run	Not done

7.5 Discussion and Limitations

What the results support. A small CNN and a modified ResNet18 can separate real CIFAR-10 photographs from Stable Diffusion v1.4 images at 32×32 resolution with 95.7% and about 97.7% validation accuracy, and the ROC-AUC of 0.9912 shows that the scores rank the two classes well across thresholds. The pipeline is verified, reproducible and resumable.

What they do not support. The validation split comes from the same generator, the same resolution and the same class prompts as the training data, so it measures in-distribution performance. Recent benchmarks show that detectors can fall sharply when the generator changes or the image is compressed or re-shared [12], [14]. The original study also found that the classifier relies on small, sometimes background, artefacts [1]; such artefacts may be specific to Stable Diffusion v1.4 and to the 512-to-32 resizing step. The figures in this chapter should therefore be read as a strong baseline result on a controlled benchmark and not as a general accuracy for detecting AI-generated images.

Reading the results. The next scientifically meaningful step is not a higher CIFAKE score. It is to report the official test-set result and to measure how much accuracy is lost on unseen generators and degraded images.

Challenges and Learning Outcomes

Practical problems met while building the system, the solutions adopted and the skills gained

8.1 Challenges Faced

Table 8.1 Main challenges and their effect.

Challenge	Effect on the project
Colab sessions can disconnect and their disk is temporary	Trained models and figures can be lost with the runtime
GPU allocation on the free tier is not guaranteed	Training must check for a GPU and be prepared to resume
120,000 small image files	Slow to copy to Drive; data loading dominates epoch time
Very small 32×32 inputs	Fine detail is lost, so the benchmark is easier and less realistic than real-world use
Validation versus test confusion	A strong validation score is easily reported as if it were a test result
Choosing a checkpoint	The best accuracy and the best loss occur at different epochs (ResNet18: epochs 11 and 9)
Interpreting high scores	Accuracy alone does not reveal which errors occur or whether the model generalises

8.2 Solutions Implemented

- **Persistence:** the best model state and history are written to Google Drive after each improving epoch, and a separate cell reloads them.
- **Runtime discipline:** the GPU is checked at the start of every session, and the raw dataset stays on the fast local disk instead of Drive.
- **Verification first:** class counts and split sizes are printed before training begins.
- **Consistent selection rule:** checkpoints are chosen by validation loss, and the report states the accuracy at that checkpoint as well as any higher peak.
- **Transparent evaluation:** the confusion matrix is reported so that every metric can be recomputed by hand, and the official test set is kept untouched.
- **Honest scope:** experiments that were not run are described as future work, not as results.

8.3 Technical Skills Learned

Table 8.2 Skills developed during the project.

Area	Learning outcome
Computer vision	Working with image folders, RGB tensors, normalisation and augmentation
Deep learning	Convolution blocks, residual connections, logits and binary cross-entropy
PyTorch	Dataset and DataLoader pipelines, model classes, training loops, checkpointing
Model evaluation	Confusion matrices, precision, recall, F1-score and ROC-AUC, and how to read them together
Experiment design	Stratified seeded splits, early stopping, controlled model comparison
Cloud notebooks	Working within Colab runtime limits, Drive persistence, GPU checks
Deployment basics	Exposing a trained classifier through a Gradio interface
Research reading	Connecting a benchmark paper to newer work on generalisation and robustness

The most important lesson was methodological. A high validation score answers a narrow question, and the wider forensic question depends on data diversity, image processing and evidence beyond the pixels. Learning to state what a number does and does not show turned out to be as valuable as the model itself.

Future Enhancements

Moving from a controlled benchmark towards a broader forensic system

9.1 Proposed Improvements

Official test-set evaluation. The first and simplest improvement is to evaluate the saved checkpoints once on the 20,000-image test set, without changing any setting after seeing the result. This gives a number that can be compared fairly with the published study.

Cross-generator evaluation. The notebook already reserves an *ood_data* folder for images from other generators. The proposed experiment freezes the trained detector and tests it on unseen generators, with the same 512-to-32 bilinear preprocessing so that resolution is not a confounding factor. GenImage defines this kind of task and Community Forensics shows why generator diversity matters [12], [13].

Robustness to image processing. A sweep of JPEG compression, blur and resizing should measure how quickly accuracy degrades, reporting both the absolute score and the change from the clean images. RRDataset shows that transmission and re-digitisation can change the statistics on which a detector relies [14].

Explainability and error analysis. The grad-cam library is already installed in the notebook. Applying Grad-CAM [16] to correct and incorrect predictions would show whether the model attends to meaningful forensic cues, as the original study found [1]. The aim is error analysis and not just attractive heat maps.

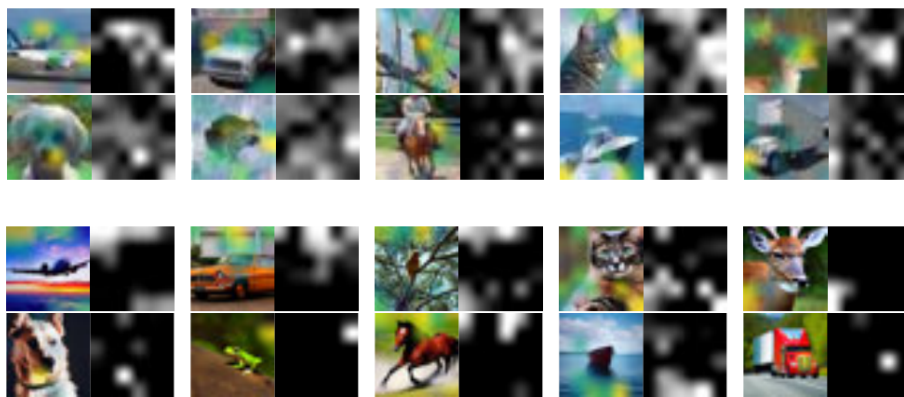


Figure 9.1 Grad-CAM overlays and heat maps from the original CIFAKE study.

Source: Bird and Lotfi [1], Fig. 6. Top: real images. Bottom: AI-generated images. Brighter pixels contribute more to the decision.

Frequency and reconstruction evidence. Spectral analysis and reconstruction-error features are complementary signals that do not depend on object identity [7], [9], [10], [11]. Combining them with the CNN score could reduce reliance on any single artefact.

Mitigation by augmented training. If the robustness sweep shows large drops, the detector can be retrained with compression, blur and resize augmentation, following the approach shown to help transfer in earlier work [5].

Calibration. A probability is not a certainty. Future work should measure calibration, study threshold choice, and present outputs as model confidence rather than as a statement of authenticity, especially for images outside the training distribution.

9.2 Additional Features

- Batch inference for folders of images, with a downloadable table of scores.
- Heat-map output beside the prediction in the interface, so users can see where the evidence lies.
- A model card recording the training data, metrics, known limits and intended use.
- A programmatic API endpoint for use by other applications.
- A provenance panel that reports whether content credentials or a watermark are present [21], [22].

9.3 Scalability Opportunities

Scaling can proceed along four axes: data (larger and more diverse datasets such as GenImage and Community Forensics), model (patch-based processing or pretrained transformer backbones at higher resolution), compute (mixed-precision training and a faster input pipeline, given that data loading appears to dominate epoch time) and deployment (moving from a temporary Colab link to a permanent hosted service). Figure 9.2 shows how additional evidence channels could be fused with the current classifier, and Table 9.1 lists the enhancements by priority.

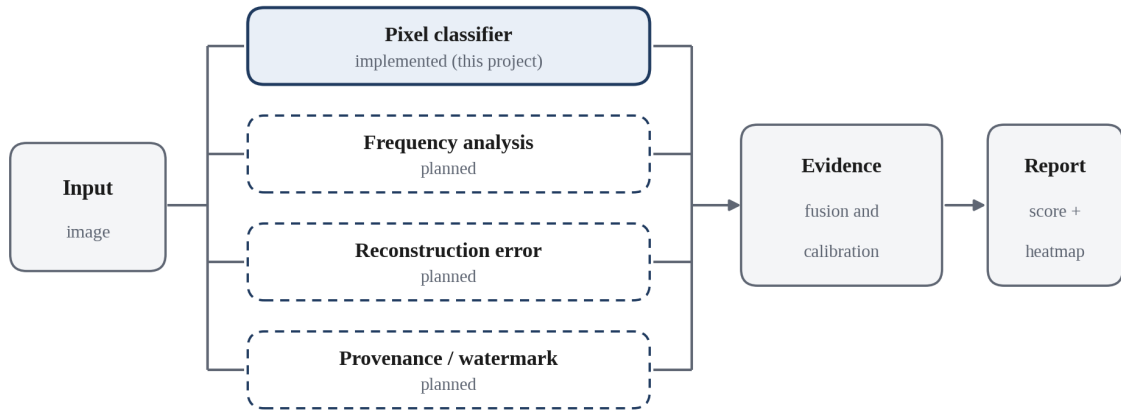


Figure 9.2 Proposed evidence-fusion extension of the current classifier.

Source: project roadmap. Solid outline: implemented; dashed outline: planned.

Table 9.1 Enhancement roadmap.

Enhancement	Expected benefit	Priority
Official test-set evaluation	Comparable, unbiased final accuracy	High
Cross-generator benchmark	Measures generalisation beyond Stable Diffusion v1.4	High
JPEG, blur and resize sweep	Quantifies degradation under common image operations	High
Grad-CAM and error review	Shows what the classifier relies on	High
Spectral and reconstruction features	Adds complementary forensic evidence	Medium
Higher-resolution training	Preserves fine detail lost at 32×32	Medium
Permanent hosted deployment	Access beyond a temporary Colab link	Medium
Provenance and watermark checks	Evidence independent of pixels	Medium

Conclusion

Summary of the completed work and its significance

10.1 Summary of Work Completed

The project built an end-to-end detection pipeline on the CIFAKE benchmark: the dataset was downloaded and verified, a compact CNN following the published topology was trained and evaluated, and a ResNet18 adapted to 32×32 inputs was added for comparison. Drive checkpointing, evaluation and a Gradio interface completed the system.

PaperCNN reached 95.68% validation accuracy with 96.44% precision, 94.86% recall, an F1-score of 95.64% and a ROC-AUC of 0.9912. ResNet18 peaked at 97.77% validation accuracy.

10.2 Achievements

- Built a reproducible and resumable CIFAKE experiment in Google Colab.
- Compared two detector architectures under identical data conditions with auditable evaluation outputs.
- Placed the results in the context of current research and defined a roadmap towards a more realistic forensic detector.

10.3 Project Impact

The value of the work lies in turning a published research idea into a working, checked and documented engineering pipeline, and then stating clearly how far its results can be trusted. It shows that a simple classifier can perform strongly on a controlled benchmark, and why broader evaluation is still needed. The notebook can serve as the base for the extensions in Chapter 9.

10.4 Final Remark

CIFAKE is a good starting point because its construction is transparent and its task is easy to reproduce. The meaningful next step is not to claim that the detector finds every AI-generated image, but to test what it has learned against new generators and degraded images. Moving from benchmark accuracy to evidence quality is the central lesson of this project.

References

- [1] J. J. Bird and A. Lotfi, “CIFAKE: Image classification and explainable identification of AI-generated synthetic images,” *IEEE Access*, vol. 12, pp. 15642–15649, 2024, doi: 10.1109/ACCESS.2024.3356122.
- [2] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.
- [3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 10684–10695.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851.
- [5] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, “CNN-generated images are surprisingly easy to spot... for now,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 8695–8704.
- [6] U. Ojha, Y. Li, and Y. J. Lee, “Towards universal fake image detectors that generalize across generative models,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 24480–24489.
- [7] R. Corvi *et al.*, “On the detection of synthetic images generated by diffusion models,” arXiv:2211.00680, 2022.
- [8] Z. Sha, Z. Li, N. Yu, and Y. Zhang, “DE-FAKE: Detection and attribution of fake images generated by text-to-image generation models,” arXiv:2210.06998, 2022.
- [9] Z. Wang *et al.*, “DIRE for diffusion-generated image detection,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 22445–22455.
- [10] Q. Bammey, “Synthbuster: Towards detection of diffusion model generated images,” *IEEE Open J. Signal Process.*, vol. 5, pp. 1–9, 2024.
- [11] J. Ricker, D. Lukovnikov, and A. Fischer, “AEROBLADE: Training-free detection of latent diffusion images using autoencoder reconstruction error,” in *Proc. IEEE/CVF CVPR*, 2024, pp. 9130–9140.
- [12] M. Zhu *et al.*, “GenImage: A million-scale benchmark for detecting AI-generated image,” in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.
- [13] J. Park and A. Owens, “Community Forensics: Using thousands of generators to train fake image detectors,” in *Proc. IEEE/CVF CVPR*, 2025, pp. 8245–8257.
- [14] C. Li *et al.*, “Bridging the gap between ideal and real-world evaluation: Benchmarking AI-generated image detection in challenging scenarios,” in *Proc. IEEE/CVF ICCV*, 2025, pp. 20379–20389.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778.
- [16] R. R. Selvaraju *et al.*, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.
- [17] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, arXiv:1412.6980.

- [18] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.
- [19] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [20] A. Abid *et al.*, “Gradio: Hassle-free sharing and testing of ML models in the wild,” arXiv:1906.02569, 2019.
- [21] Coalition for Content Provenance and Authenticity, “C2PA technical specification.” [Online]. Available: <https://c2pa.org>
- [22] Google DeepMind, “SynthID.” [Online]. Available: <https://deepmind.google/technologies/synthid/>
- [23] S. J. Nightingale, K. A. Wade, and D. G. Watson, “Can people identify original and manipulated photos of real-world scenes?” *Cogn. Res., Princ. Implic.*, vol. 2, no. 1, 2017.
- [24] D. Gunning *et al.*, “XAI—Explainable artificial intelligence,” *Sci. Robot.*, vol. 4, no. 37, 2019, Art. no. eaay7120.
- [25] J. J. Bird, “CIFAKE: Real and AI-generated synthetic images,” Kaggle dataset. [Online]. Available: <https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images>

Epoch-wise Training Records

Values parsed from the executed notebook output

Table A.1 PaperCNN, 18 epochs (early stopping).

Epoch	Training loss	Validation loss	Validation accuracy	Time (s)	Note
1	0.3182	0.2173	91.11%	170.2	
2	0.2037	0.1992	91.80%	62.6	
3	0.1702	0.1786	92.87%	63.4	
4	0.1570	0.1805	93.05%	63.0	
5	0.1445	0.1432	94.39%	64.2	
6	0.1390	0.1362	94.43%	63.9	
7	0.1294	0.1690	93.44%	66.6	
8	0.1205	0.1294	95.06%	65.7	
9	0.1147	0.1231	95.18%	66.6	
10	0.1088	0.1245	95.25%	64.9	
11	0.1010	0.1405	94.92%	63.7	
12	0.0960	0.1366	94.82%	65.2	
13	0.0914	0.1228	95.68%	63.9	best validation loss, checkpoint saved
14	0.0842	0.1257	95.38%	65.1	
15	0.0825	0.1368	95.29%	65.6	
16	0.0777	0.1407	95.18%	67.6	
17	0.0724	0.1251	95.67%	65.5	
18	0.0683	0.1271	95.65%	63.3	

Table A.2 ResNet18, 14 epochs (early stopping).

Epoch	Training loss	Validation loss	Validation accuracy	Time (s)	Note
1	0.1555	0.1423	94.51%	85.7	
2	0.0947	0.1194	95.51%	83.7	
3	0.0765	0.0888	96.85%	87.7	
4	0.0638	0.0837	97.01%	86.4	
5	0.0588	0.0932	96.91%	86.6	
6	0.0469	0.0840	97.08%	85.4	
7	0.0379	0.1264	96.11%	85.4	
8	0.0362	0.0710	97.71%	85.4	
9	0.0301	0.0677	97.75%	85.9	best validation loss, checkpoint saved
10	0.0263	0.0745	97.76%	86.7	
11	0.0229	0.0739	97.77%	84.8	peak accuracy (not saved)
12	0.0213	0.0824	97.38%	85.5	
13	0.0194	0.0799	97.69%	85.1	
14	0.0169	0.0770	97.60%	85.4	